

Assignment 3 | Bridge Pattern

Astana IT University | School of Software Engineering

Course: ShP-2216 Software Design Patterns | 2026-2027 | Yerassyl Bekenov

Students: 6B06102 Software Engineering | Year 2, Trimester 4

Format: Individual | Java, JDK 17 | English | 15% of the 1st Attestation grade

Submission: 04.10.2026, 23:59, end of Week 4.

Defense Deadline: Your group's last practice class of Week 5.

1 Your task

Build a small Java application using Bridge to connect two independently varying class hierarchies. Demonstrate every required combination, replace an implementation on the same object at runtime, and add one implementation without changing the existing hierarchies.

Outcomes: explain Bridge and compare it with Adapter (CLO 1); implement and test it (CLO 2); defend your design (CLO 3). Every topic uses the same checks and rubric.

2 Choose one topic

Choose one option A-D and use the listed type names. A1/A2 are refined abstractions; I1/I2 are initial implementations; I3 is added in the extension step.

Option	Abstraction and subclasses	Implementor and implementations
A Drawing	Shape A1: Circle; A2: Square	Renderer I1: VectorRenderer; I2: RasterRenderer I3: AsciiRenderer
B Notifications	Notification A1: Reminder; A2: UrgentAlert	Channel I1: EmailChannel; I2: SmsChannel I3: PushChannel
C Reports	Report A1: AttendanceReport A2: GradeReport	Formatter I1: TextFormatter; I2: HtmlFormatter I3: MarkdownFormatter
D Remote controls	Remote A1: BasicRemote; A2: QuietRemote	Device I1: TvDevice; I2: RadioDevice I3: ProjectorDevice

Minimum behavior for your topic

A. Circle uses radius 2; Square uses side 3. Each renderer returns a visibly different rendering description containing the shape and its dimension. Text simulation is sufficient.

B. Reminder creates a normal reminder; UrgentAlert creates a message marked URGENT. Preserve the message content. Show an email envelope, a single-line SMS, or a push notification envelope, depending on the channel.

C. AttendanceReport calculates 3 of 4 attended sessions as 75%; GradeReport calculates the average of 70, 80, and 90 as 80. Format the calculated content as plain text, HTML, or Markdown.

D. BasicRemote powers on a device at volume 30; QuietRemote powers it on at volume 5. Each device returns its type, power state, and resulting volume. A single apply-settings operation may implement this simulation.

3 Implementation requirements

- Use an abstract base class on the abstraction side and two subclasses with the distinct behavior in Section 2. Give each abstraction an ID and retain its domain data, such as size, message, report data, or volume preset.
- The base class must hold an Implementor-interface reference supplied through its constructor. Expose a public `String execute()` operation and a `setImplementation(...)` method whose parameter is that interface.
- Define the low-level operations in the Implementor interface. Implement them in I1 and I2. Abstraction-side behavior must call those operations through the stored interface reference.
- Use composition to connect the two hierarchies. The abstraction classes must not construct concrete implementors, cast to them, or branch on their concrete types. Do not create a subclass for every A/I combination.
- Use JDK classes only. Return results as strings so the demo can check them. Simulate rendering, delivery, formatting, or device control locally; a GUI, network connection, database, and real hardware are not required.

4 Add an independent implementation

First complete the two-by-two solution and runtime switching with I1 and I2, then commit that working version. Record its hash as the base commit. Next add I3 from your topic and commit the extension. For this step, the only Java source changes allowed are the new I3 class and updates to Main for the demonstration. You may also update sources.txt and documentation.

Keep the existing abstraction classes, Implementor interface, I1, and I2 unchanged. Save the source diff as `extension.diff` after committing I3. Replace `BASE_COMMIT` below with the recorded hash:

```
git diff BASE_COMMIT HEAD -- src > extension.diff
```

5 Required demonstration checks

Implement `Main --demo` to run T1-T7 in this order without interactive input. Use fixed sample data. Each check must call the real application and compare the actual result with the expected result or state; calculate PASS or FAIL from that comparison.

Check	Setup or action	Required evidence
T1	A1 with I1	Correct A1 behavior and I1 output.
T2	A1 with I2	Same A1 data; correct I2 output.
T3	A2 with I1	Correct A2 behavior and I1 output.
T4	A2 with I2	Same A2 data; correct I2 output.
T5	On one A1 object, run with I1, replace it with I2, then run again.	Reference equality using <code>==</code> ; unchanged ID and domain data; actual results before and after the switch.
T6	A1 with the new I3	Correct A1 behavior through I3.
T7	A2 with the new I3	Correct A2 behavior through I3.

For T5, retain a reference to the original object and compare it with the reference used after the switch. Matching IDs alone do not prove object identity. Preserve abstraction-side data; the new implementor may have its own separate internal state.

6 Prepare a submission that can be checked quickly

Upload one ZIP named `Assignment3_Group_Surname_Name.zip` to Moodle. Put the following items directly at the ZIP root. In the Moodle submission text, provide the GitHub repository URL and the exact submitted commit hash. The ZIP must match that source version.

Item	Contents
src/	All Java source files. Main.java must be in the default package; other classes may use packages.
sources.txt	One relative Java source path per line, including src/Main.java and the I3 file.
README.md	Your topic, role map, run commands, expected outcomes, and base commit hash.
report.pdf	A concise report in English, normally 2-3 pages, as described below.
demo-output.txt	Console output captured from the submitted program running all seven checks.
extension.diff	Source diff from the working I1/I2 version to the committed I3 extension.

Standard build and run commands

From the extracted project folder, these two commands must compile and run the assignment without an IDE or extra dependencies:

```
javac --release 17 -encoding UTF-8 -d out "@sources.txt"
java -cp out Main --demo
```

For each T1-T7 check, print its ID, PASS or FAIL, participating classes, and the actual result. A failed check must also show what was expected. Run all checks and finish with a computed summary such as SUMMARY: 7/7 PASS. Keep output compact. Do not hard-code successful test labels.

Example of the output format only; choose the exact expected strings for your topic in README:

```
T1 PASS | Circle + VectorRenderer | result=VECTOR circle radius=2
T5 PASS | sameObject=true | stateUnchanged=true
before=<actual I1 result> | after=<actual I2 result>
SUMMARY: 7/7 PASS
```

README review map

At the top of README, state your name, group, topic letter, repository URL, and base commit hash. Add a compact table mapping Abstraction, A1, A2, Implementor, I1, I2, I3, and Client to their class names and source paths. Also point to the bridge field, `execute()`, `setImplementation(...)`, and the T5 check. Include the two commands above and expected results for T1-T7.

Short report

- State the full assignment title, your name, group, and topic. Explain the two independent dimensions and why Bridge fits.
- Include one readable UML class diagram with both hierarchies, all three implementors, the client, relevant methods, and the interface-typed reference connecting the hierarchies. It must match the final code.
- Provide five short annotated code excerpts showing: separation of responsibilities, meaningful names, small focused methods, avoidance of duplication, and encapsulation of state. Explain the benefit of each.
- Explain how I3 was added, what stayed unchanged, one trade-off of Bridge, and how its intent differs from Adapter. Cite the sources you used.

7 Grading

Total: 100 points = report 10 + code 40 + defense 50. Each report or code check earns its stated points when fully met, otherwise zero. Partial credit is the sum of completed checks; all topics use this rubric.

Report 10 points

Criterion	Max	Award points for these checks
R1 UML	4	1 each: correct abstraction hierarchy; interface with all three implementors; the interface-typed bridge link; methods and client dependencies matching the code.
R2 Explanation	4	1 each: two dimensions and Bridge rationale; correct comparison with Adapter; five annotated Clean Code excerpts with benefits; extension explanation and one specific trade-off.
R3 Submission evidence	2	1: report identification, topic, references, and submitted repository/commit are provided. 1: demo-output.txt records actual T1-T7 outcomes, including any failures, and the extension diff is present.

Working and clean code 40 points

Criterion	Max	Award points for these checks
C1 Bridge structure	12	3: the abstract base stores an interface-typed reference supplied through its constructor. 3: A1/A2 extend the base and implement their distinct behavior through the bridge. 3: I1/I2 implement the interface and provide the required distinct behavior. 3: abstraction classes delegate through the interface, with no concrete implementor dependencies, type checks, casts, or combination subclasses.
C2 Combinations	8	2 per correct T1-T4: required domain content and the selected implementation. A printed PASS without correct underlying behavior earns zero.
C3 Runtime switch	6	2: T5 proves the same abstraction object using ==. 2: the second execute() call uses I2 after runtime replacement. 2: the abstraction ID and domain data stay unchanged.
C4 Extension	6	2 per correct check T6 and T7. 2: the diff shows only the new I3 source and Main changed within src/; existing abstractions, interface, I1, and I2 remain unchanged.
C5 Clean Code	5	1 per practice applied in the implementation: separated responsibilities; meaningful names; focused methods; no duplicated workflow or formatting logic; encapsulated state. The report excerpts must identify concrete examples.
C6 Reproducibility	3	1: the standard commands compile and run the submitted source. 1: --demo runs without input and produces T1-T7 plus a calculated summary. 1: README contains the source role map, method locations, commands, and expected outcomes.

If the program cannot run, checks requiring execution earn zero; assess the remaining structure and documentation from the available evidence. Score each listed check once. Extra features do not replace missing required checks.

8 Individual defense

Prepare a five-minute defense. Open README, the UML diagram, Main, and the abstraction class before your turn. Use the submitted version. You must be able to explain, in your own words, how your own submitted code works; being unable to do so for any part of it, even partially, can cost significant points regardless of whether that code runs correctly (see the code-comprehension deduction after the Defense 50 points table). The review follows this order:

Time	What you demonstrate
Minute 1	Run --demo; show the seven results and any failures.
Minute 2	Use the role map and code to identify the two hierarchies and trace one delegated call.
Minute 3	Show T5 and explain which reference changes and which object and data stay the same.
Minute 4	Show extension.diff; explain independent extension and the difference from Adapter.
Minute 5	Explain one method chosen by the instructor and predict the result of one small input or implementation change.

The instructor may select a different T1-T7 case or sample value to verify your understanding. You must explain the result using your own code.

Defense 50 points

Each checkpoint below is worth 5 points. Award 5 for an accurate explanation supported by the relevant code or evidence; 3 when the main idea is correct but the code link or one required detail is missing; 0 when the explanation is absent or incorrect. Add the checkpoint scores within each row.

Criterion	Max	Five point checkpoints
D1 Roles and delegation	20	Four checkpoints: identify Abstraction and A1/A2; identify Implementor and I1/I2/I3; trace execute() into an implementor operation; explain why the two dimensions are separate and linked by composition.
D2 Runtime switch	10	Two checkpoints: explain the implementation-reference update and same-object proof; explain why the domain state is preserved while the implementation-specific result changes.
D3 Design decisions	10	Two checkpoints: explain the I3 extension using the diff and the unchanged contract; distinguish Bridge from Adapter using the intent and your application.
D4 Own code	10	Two checkpoints: explain the purpose, inputs, and main steps of an instructor-selected method; correctly predict and justify the effect of a small input or implementation change.

Code comprehension deduction. If, at any point during the defense, a student cannot explain how a part of their own submitted code works, or shows only a partial or superficial understanding of it, the instructor may deduct up to 20 points from the total Defense score, in addition to the D1-D4 checkpoints above. This applies to any part of the submission the instructor chooses to probe, not only the method selected for D4, and regardless of whether that code runs correctly, compiles, or passed T1-T7. The deduction reflects the requirement in Section 8 that the student can explain, in their own words, the code they submitted as their own work; it is not a substitute for the D1-D4 checkpoints and does not replace them. The instructor determines the size of the deduction based on how much of the code is not understood and how central that part is to the Bridge structure (Sections 2-3).

Examples: three correct T1-T4 checks earn 6/8 in C2. One complete and one partially supported D2 answer earn 8/10. Add R1-R3, C1-C6, and D1-D4, apply the code-comprehension deduction above if it applies, subject to the course rules below.

9 Course rules

Defense is mandatory. Failure to defend results in zero for the entire assignment under the syllabus. Late submissions receive zero unless an extension has been approved under the course and university rules for an exceptional circumstance.

Write and understand your own code and report; cite sources. Conceptual discussion is allowed. AI tools may be used only for reference examples and clarification as allowed by the syllabus.

Resources: Lecture 4, Bridge Pattern; course syllabus 2026-2027; Bridge reading in Moodle. Use PlantUML, UMLet, or another UML tool for the diagram.